

Java Pattern Programs — Practice Set

22 loop-based pattern problems, ordered easy to hard. Each entry shows the problem, the expected output for $n = 5$, and the Java solution. Cover the answer and try it yourself first.

Level 1 — Basics (single condition)

1. Square

Expected output ($n = 5$):

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

2. Right Triangle

Expected output ($n = 5$):

```
*
* *
* * *
* * * *
* * * * *
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

3. Inverted Right Triangle

Expected output ($n = 5$):

```
* * * * *
* * * *
* * *
* *
*
```

Java solution:

```
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

4. Left Triangle (right-aligned)

Expected output (n = 5):

```
      *
     * *
    * * *
   * * * *
  * * * * *
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= i; j++) System.out.print("* ");
    System.out.println();
}
```

5. Inverted Left Triangle

Expected output (n = 5):

```
* * * * *
 * * * *
  * * *
   * *
    *
     *
```

Java solution:

```
for (int i = n; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= i; j++) System.out.print("* ");
    System.out.println();
}
```

Level 2 — Spaces + Stars Together

6. Pyramid (centered triangle)

Expected output (n = 5):

```
  *
 ***
*****
*****
*****
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}
```

7. Inverted Pyramid

Expected output (n = 5):

```
*****
*****
*****
***
*
```

Java solution:

```
for (int i = n; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}
```

8. Diamond

Expected output (n = 5):

```
  *
 ***
*****
*****
*****
*****
***
*
```

Java solution:

```
// upper half
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}
// lower half
for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}
```

Level 3 — Numbers Instead of Stars

9. Row Number Repeated

Expected output (n = 5):

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(i + " ");
    }
    System.out.println();
}
```

10. Column Numbers (1, 1 2, 1 2 3...)

Expected output (n = 5):

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j + " ");
    }
    System.out.println();
}
```

11. Floyd's Triangle (continuous numbering)

Expected output (n = 5):

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Java solution:

```
int num = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(num++ + " ");
    }
    System.out.println();
}
```

12. Increasing then Decreasing (1 2 3 2 1)

Expected output (n = 5):

```
1
1 2 1
1 2 3 2 1
```

1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1

Java solution:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) System.out.print(j + " ");  
    for (int j = i - 1; j >= 1; j--) System.out.print(j + " ");  
    System.out.println();  
}
```

Level 4 — Characters

13. Letter Triangle (A B C D...)

Expected output (n = 5):

```
A
A B
A B C
A B C D
A B C D E
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        char ch = (char) (64 + j);
        System.out.print(ch + " ");
    }
    System.out.println();
}
```

14. Repeating Letter per Row (A / B B / C C C)

Expected output (n = 5):

```
A
B B
C C C
D D D D
E E E E E
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    char ch = (char) (64 + i);
    for (int j = 1; j <= i; j++) {
        System.out.print(ch + " ");
    }
    System.out.println();
}
```

Level 5 — Hollow Shapes (need if inside loop)

15. Hollow Square

Expected output ($n = 5$):

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == 1 || i == n || j == 1 || j == n) System.out.print("* ");
        else System.out.print(" ");
    }
    System.out.println();
}
```

16. Hollow Right Triangle

Expected output ($n = 5$):

```
*
* *
*  *
*   *
* * * * *
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        if (j == 1 || j == i || i == n) System.out.print("* ");
        else System.out.print(" ");
    }
    System.out.println();
}
```

17. Hollow Pyramid

Expected output ($n = 5$):

```
  *
 * *
*   *
*   *
*****
```

Java solution:

```
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) {
        if (j == 1 || j == 2 * i - 1 || i == n) System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

18. Hollow Diamond

Expected output ($n = 5$):

```

      *
    * *
  *   *
*     *
*****
*     *
*   *
* *
*

```

Java solution:

```

// upper half (hollow pyramid)
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) {
        if (j == 1 || j == 2 * i - 1 || i == n) System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
// lower half (hollow inverted pyramid)
for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) {
        if (j == 1 || j == 2 * i - 1 || i == n) System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}

```


Level 6 — Boss Patterns

19. Pascal's Triangle (real nCr version)

Expected output (n = 5):

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Java solution:

```
for (int i = 0; i < n; i++) {
    int val = 1;
    for (int s = 0; s < n - i - 1; s++) System.out.print(" ");
    for (int j = 0; j <= i; j++) {
        System.out.print(val + " ");
        val = val * (i - j) / (j + 1);
    }
    System.out.println();
}
```

20. Butterfly Pattern

Expected output (n = 5):

```
*          *
**         **
***        ***
****       ****
*****      *****
*****      *****
****       ****
***        ***
**         **
*          *
```

Java solution:

```
// upper half
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) System.out.print("*");
    for (int s = 1; s <= 2 * (n - i); s++) System.out.print(" ");
    for (int j = 1; j <= i; j++) System.out.print("*");
    System.out.println();
}
// lower half
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) System.out.print("*");
    for (int s = 1; s <= 2 * (n - i); s++) System.out.print(" ");
    for (int j = 1; j <= i; j++) System.out.print("*");
    System.out.println();
}
```

21. X Pattern (two diagonals crossing)

Expected output (n = 5):

```
*          *
*          *
*         *
*        *
*       *
*      *
*     *
```

```

    * *
  *   *
*     *
*       *

```

Java solution:

```

int size = 2 * n - 1;
for (int i = 1; i <= size; i++) {
    for (int j = 1; j <= size; j++) {
        if (j == i || j == size - i + 1) System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}

```

22. Sandglass / Hourglass Pattern

Expected output ($n = 5$):

```

*****
 *
*****
 *
 *
 *
 *
 *
 *
 *
*****
*****

```

Java solution:

```

// upper inverted triangle
for (int i = n; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}
// lower triangle (skip row 1, already printed as the tip)
for (int i = 2; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) System.out.print(" ");
    for (int j = 1; j <= 2 * i - 1; j++) System.out.print("*");
    System.out.println();
}

```

How to use this: for each pattern, write the expected output on paper first, then write the code yourself before checking the solution above. n is set to 5 throughout — re-test each solution with a different n (e.g. 7) to confirm you actually understand the loop logic and didn't just memorize it.